METODOLOGIA AIDD & OUROBOROS

Manual Prático do Arquiteto de Pipelines

O Método Definitivo para Descompor Qualquer Problema em Fluxos Determinísticos, Neutralizar os Vícios da Inteligência Artificial e Criar Sistemas que Nunca Falham.

Autor: Engenharia AIDD Ouroboros
Propósito: Guia de Estudo, Formação de Raciocínio & Caderno de Fichas de Preenchimento
Versão: 1.0 (Edição Canônica)

MÓDULO 1: Por Que Nos Sentimos Perdidos? (A Psicologia do Arquiteto)

Quando tentamos criar um sistema ou automação moderna, é comum sentirmos que "quanto mais pensamos, menos entendemos". Esse sentimento não ocorre por falta de inteligência. Ele ocorre por um fenômeno neurocientífico e técnico chamado **Sobrecarga de Espaço de Estados**.

Quando você olha para o objetivo final — por exemplo, *"pegar um vídeo do YouTube e transformá-lo num pipeline completo com código e animações"* —, o cérebro humano tenta processar o início, o meio, o fim, os erros de digitação, a linguagem de programação, a API que pode cair e o prompt da IA **ao mesmo tempo**.

> **O Segredo dos Grandes Engenheiros:** Ninguém resolve um sistema complexo inteiro de uma vez. O trabalho do Arquiteto não é programar; é **cortar o elefante em bifes tão pequenos que cada um deles se torne óbvio**.

Os Dois Vícios Estruturais das LLMs

Para piorar a confusão, quando pedimos ajuda a uma Inteligência Artificial sem rédeas curtas, ela ativa dois vícios de fábrica que nos desorientam ainda mais:

> **1. A Ânsia da Entrega Simulada:** A IA tem pavor de dizer "não sei" ou "preciso de um arquivo físico". Ela prefere inventar dados falsos (mocks), fingir que conectou numa API e cuspir um texto bonito que não funciona na prática.

> **2. O Loop de Autocorreção Cega:** Ao encontrar um erro num código sem evidência física, a IA tenta adivinhar o motivo. Ela mexe em arquivos aleatórios, quebra o que já funcionava e entra em um turbilhão sem fim de remendos.

Para anular esses dois vícios e blindar sua própria mente contra a confusão, você só precisa de um princípio: **O Princípio dos Dois Baldes**.

MÓDULO 2: O Princípio dos Dois Baldes (A Linha Divisória Sagrada)

Toda e qualquer tarefa no universo da computação pertence obrigatoriamente a um dos dois baldes abaixo. Nunca misture os dois:

BALDE 1: Determinístico (Zero IA)

Regra: Se existe uma fórmula matemática, um utilitário do sistema operacional ou uma resposta exata (0 ou 1), **a IA é estritamente proibida de raciocinar**.

- Baixar arquivos da internet
- Cortar e converter vídeos ou áudios
- Transcrever fala para texto
- Ler texto em imagens (OCR)
- Criar pastas e checar se arquivos existem
- Calcular hashes criptográficos (SHA-256)

Custo: 0 Tokens | Confiabilidade: 100%

BALDE 2: Probabilístico (Usa IA)

Regra: A IA só é acionada onde existe **julgamento humano, ambiguidade, síntese ou criação de linguagem**.

- Interpretar a didática e intenção de um autor
- Resumir e catalogar o método principal
- Redigir um roteiro em tom persuasivo
- Decompor uma metodologia em tarefas de código
- Formular hipóteses sobre dúvidas do código
- Explicar um erro para o operador humano

Custo: Tokens Pagos | Confiabilidade: Depende de Coleira

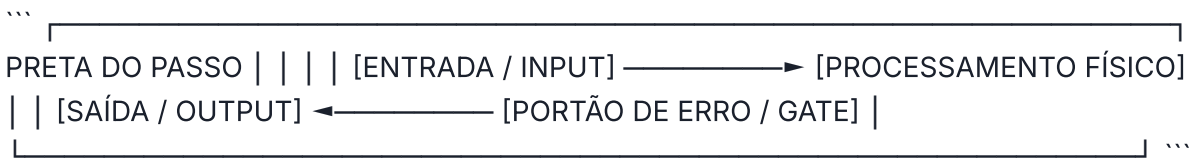
O Catálogo de Ferramentas de Prateleira (Não Reinvente a Roda)

Um dos maiores erros do iniciante é pedir para a IA criar um código do zero para algo que o mundo open-source já resolveu com perfeição há 15 anos. Decore este catálogo:

Necessidade do Pipeline	Ferramenta Consagrada (CLI / Binário)	Por Que Usar?
Baixar vídeos de qualquer site	`yt-dlp`	Padrão mundial. Baixa MP4, extrai MP3, legendas e miniaturas.
Manipular áudio e vídeo	`ffmpeg`	Corta cenas, extrai frames a cada X segundos, ajusta bitrate.
Transcrever áudio offline	`faster-whisper`	Roda no seu processador local. Transcreve com carimbo de tempo exato.
Ler texto de telas e códigos	`Tesseract OCR`	Extrai texto de imagens sem gastar nenhum token de visão.
Renderizar animações em código	`Remotion` / `Bun`	Programa vídeos em React de forma 100% determinística.
Garantir integridade de arquivos	`hashlib` (SHA-256)	Gera assinatura digital para provar que nenhum arquivo foi corrompido.

MÓDULO 3: O Contrato de Caixa-Preta (Input ➔ Processo ➔ Saída)

Um pipeline robusto é uma cadeia de **Caixas-Pretas**. Uma Caixa-Preta não se importa com quem veio antes nem com quem vem depois. Ela só se importa com o seu próprio **Contrato de 4 Elementos**:



> **OS 4 ELEMENTOS INEGOCIÁVEIS DE TODO PASSO:**
>
> 1. **ENTRADA (Input):** O que exatamente chega na mão deste passo? (Deve ser um dado físico: arquivo, caminho no disco ou parâmetro textual validado).
> 2. **PROCESSAMENTO:** Qual ferramenta executa a transformação? (Se for Balde 1, script de terminal; se for Balde 2, LLM com restrição estrita).
> 3. **PORTÃO DE ERRO (Guardrail / Circuit Breaker):** Como o sistema prova que o passo falhou *sem perguntar a opinião da IA*? (Ex: o arquivo final não existe? o código de saída é diferente de zero? a linha de texto não tem a tag obrigatória?). Se falhar, o sistema desliga a esteira na hora.
> 4. **SAÍDA (Output):** O que é gerado fisicamente no disco? Esta saída torna-se a entrada imediata do próximo passo.

MÓDULO 4: O Caso Real Dissecado Passo a Passo (Engenharia Reversa de Vídeo)

Veja agora como os 4 passos da nossa skill `aidd-rea-video-youtube` aplicam essa ciência na prática:

PASSO 1: A Coleta Física

- **Input:** Uma URL de vídeo em texto (ex: `https://youtube.com/watch?v=123`).
- **Processamento:** `100% Determinístico`. Script Python chama o binário local `yt-dlp`.
- **Portão de Erro:** Se o download retornar código de erro ou a pasta `raw/` estiver vazia, o script aborta com `sys.exit(1)`. A IA nem acorda.
- **Output:** `raw/video.mp4` e `raw/audio.mp3` salvos no disco.

PASSO 2: A Visão e Audição Local

- **Input:** Os arquivos `raw/video.mp4` e `raw/audio.mp3`.
- **Processamento:** `100% Determinístico`. 1) `faster-whisper` transcreve a fala com minutagem; 2) `ffmpeg` tira fotos a cada 10 segundos; 3) `Tesseract` lê os códigos na tela das imagens.
- **Portão de Erro:** Se `transcript.tsv` tiver tamanho 0 bytes ou a pasta `visual/` tiver menos de 3 imagens, a esteira trava.
- **Output:** `transcricao/transcript.tsv` e imagens em `visual/`.

PASSO 3: A Dissecação Técnica da Metodologia

- **Input:** O texto da transcrição com carimbo de tempo + os textos lidos pelo OCR.
- **Processamento:** `Híbrido (LLM sob Coleira de Aço)`. A IA é convocada para ler os dados e explicar a metodologia do criador.
- **Portão de Erro (A Barreira Anti-Alucinação):** A IA é **obrigada** a rotular cada conclusão:
  - `[OBSERVADO NO FRAME]` (vi na imagem X)
  - `[DITO NO ÁUDIO]` (ouvi no minuto mm:ss)
  - `[DERIVADO]` (conclusão lógica direta)
  - `[DESCONHECIDO REA]` (o autor escondeu; proibido inventar!)

  Um script Python lê o documento gerado: se houver 1 parágrafo sem evidência, o texto é recusado e apagado na hora.
- **Output:** `metodologia.md` aprovado e certificado.

PASSO 4: A Decomposição em Nano-Tickets

- **Input:** O arquivo auditado `metodologia.md`.
- **Processamento:** `Híbrido`. A IA fatia a metodologia em pequenos cartões de trabalho (nano-tickets).
- **Portão de Erro (Regra Zero Mock):** Todo ticket DEVE conter a seção `Validation Command` apontando para um script de teste real que falhe caso o código não exista. Mocks como `echo ok` são reprovados.
- **Output:** Fichas `tickets/nano-tickets/TK-01.md` e o manifesto assinado `manifest.json`.

MÓDULO 5: Como Comandar a IA Sem Deixá-la Reinventar a Roda

Use estes dois modelos de prompt prontos para assumir a postura de Diretor de Fábrica:

> **PROMPT MODELO 1: A CONSULTA DE FERRAMENTAS DE PRATELEIRA**
>
> ```
> "Estou desenhando um passo do meu pipeline para realizar: [DESCREVA O QUE QUER FAZER].
>
> NÃO escreva código novo ainda e NÃO reinvente a roda. Me responda exclusivamente:
> 1. Essa tarefa é do Balde 1 (Determinística) ou do Balde 2 (Probabilística)?
> 2. Quais são as ferramentas de prateleira, utilitários CLI ou bibliotecas consagradas de mercado que resolvem isso?
> 3. Qual é o comando de terminal padrão para executar essa tarefa?"
> ```

PROMPT MODELO 2: O ALINHAMENTO PRÉ-CÓDIGO (O ESBOÇO)

"Vamos desenhar o pipeline de [NOME DO PROJETO].
Não gere nenhuma implementação agora.
Quero que você monte uma tabela contendo as etapas com 4 colunas:
[Etapa] | [Input Físico] | [Ferramenta/Balde] | [Portão de Erro Binário] | [Output Físico].
Aguarde minha aprovação antes de propor qualquer script."

CADERNO DE TRABALHO PRÁTICO

Fichas de Desenho de Pipeline

Imprima estas páginas para rascunhar suas ideias antes de abrir o editor de código.

FICHA 1: Diagnóstico e Objetivo Central

DATA: ___/___/_____

1. NOME DO PIPELINE / IDEIA:

2. QUAL É O INPUT INICIAL BRUTO QUE O USUÁRIO ENTREGA?

3. QUAL É O PRODUTO FINAL FÍSICO QUE DEVE SAIR PRONTO NO DISCO?

4. CRITÉRIO DE SUCESSO DO CLIENTE (COMO O HUMANO SABE QUE FICOU BOM?):

FICHA 2: Matriz dos Dois Baldes (Separação Prévia)

FILTRAGEM DE RISCO

BALDE 1: O QUE É 100% DETERMINÍSTICO? (SCRIPT)

Downloads, cortes, conversões, pastas, checagens:

BALDE 2: O QUE PRECISA DE JULGAMENTO? (LLM)

Resumos, roteiros, síntese criativa, tradução:

FICHA 3: Contrato de Caixa-Preta (Para Cada Passo)

PASSO Nº: [ ]

NOME DA ETAPA:

CLASSIFICAÇÃO DA OPERAÇÃO:

☐ [Balde 1] 100% Determinístico ☐ [Balde 2] Híbrido com LLM

1. ENTRADA (INPUT FÍSICO QUE ESTE PASSO RECEBE):

2. PROCESSAMENTO (QUAL FERRAMENTA DE MERCADO OU SCRIPT EXECUTA?):

3. PORTÃO DE ERRO (COMO TESTAR SE DEU ERRO SEM PERGUNTAR PARA A IA?):

4. SAÍDA (OUTPUT FÍSICO GERADO NO DISCO PARA O PRÓXIMO PASSO):

FICHA 3: Contrato de Caixa-Preta (Para Cada Passo)

PASSO Nº: [ ]

NOME DA ETAPA:

CLASSIFICAÇÃO DA OPERAÇÃO:

☐ [Balde 1] 100% Determinístico ☐ [Balde 2] Híbrido com LLM

1. ENTRADA (INPUT FÍSICO QUE ESTE PASSO RECEBE):

2. PROCESSAMENTO (QUAL FERRAMENTA DE MERCADO OU SCRIPT EXECUTA?):

3. PORTÃO DE ERRO (COMO TESTAR SE DEU ERRO SEM PERGUNTAR PARA A IA?):

4. SAÍDA (OUTPUT FÍSICO GERADO NO DISCO PARA O PRÓXIMO PASSO):